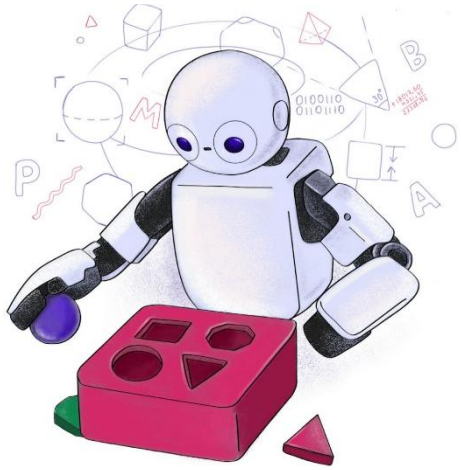
C24 - Inteligência Artificial:
k-Means



Inatel

Felipe A. P. de Figueiredo
felipe.figueiredo@inatel.br

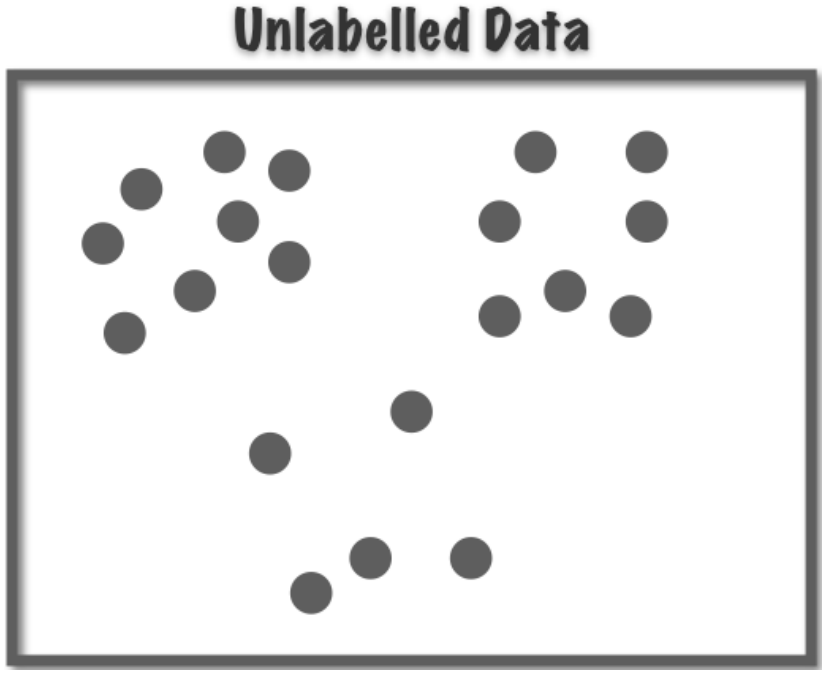
Recapitulando

- Até o momento, todos os algoritmos que aprendemos seguiam o paradigma do **aprendizado supervisionado**.
- Hoje, falaremos sobre ***clustering***, que são algoritmos de **aprendizado não-supervisionado**.
  - **Não temos rótulos**, apenas atributos.
- O objetivo do *clustering* é formar **grupos (*clusters*)** de amostras segundo sua **semelhança**.
- Nesta aula, aprenderemos o algoritmo chamado de **k-Médias** (ou ***k-Means***, em inglês) que é um dos algoritmos mais simples e populares de ***clustering***.
  - **OBS.:** Agrupamentos, grupos ou *clusters* são sinônimos neste contexto.



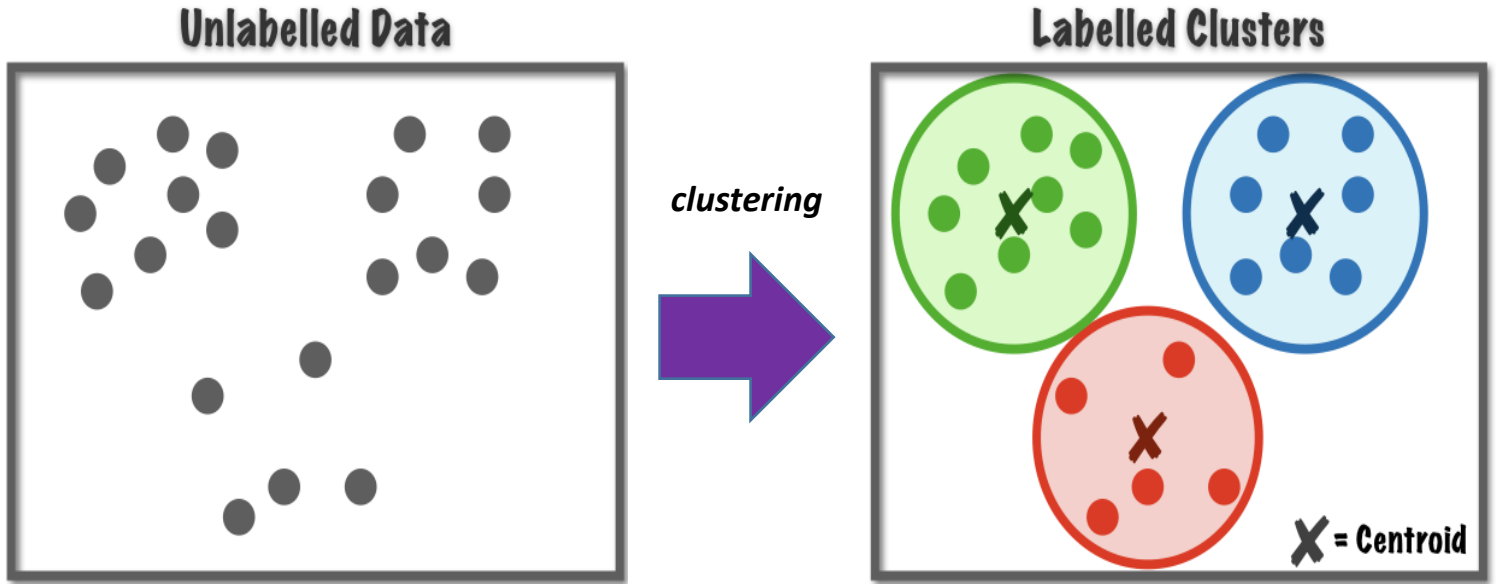
A **clusterização** organiza dados como na figura: cada exemplo fica no **grupo** ao qual é **mais próximo**, e assim formam-se os *clusters*.

Motivação



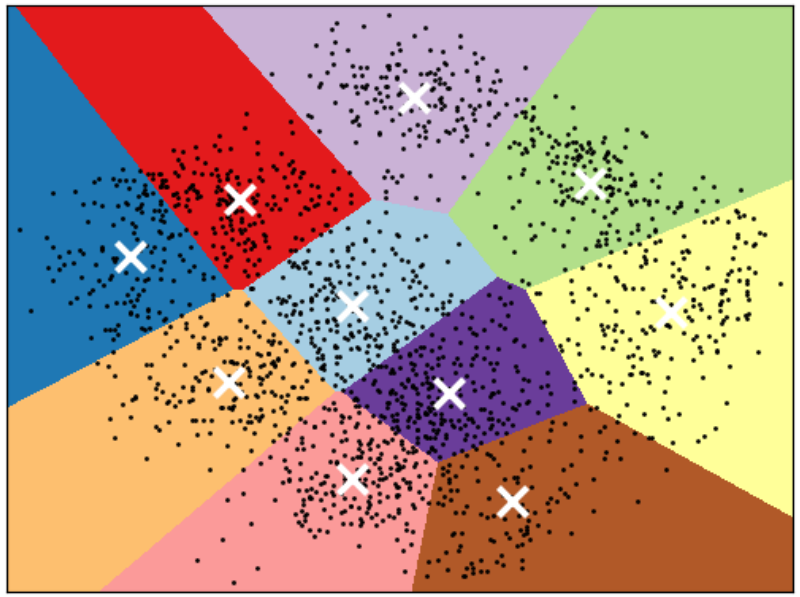
Conseguimos extrair informação útil dos dados quando **não temos as classes** (i.e., rótulos) a que eles pertencem?

Motivação



- O **aprendizado não-supervisionado** **extrai a estrutura de dados sem rótulos**.
- Assim como organizamos o mundo em **categorias por semelhança** (frutas, ferramentas), o *clustering* **agrupa dados por similaridade, sem supervisão**.

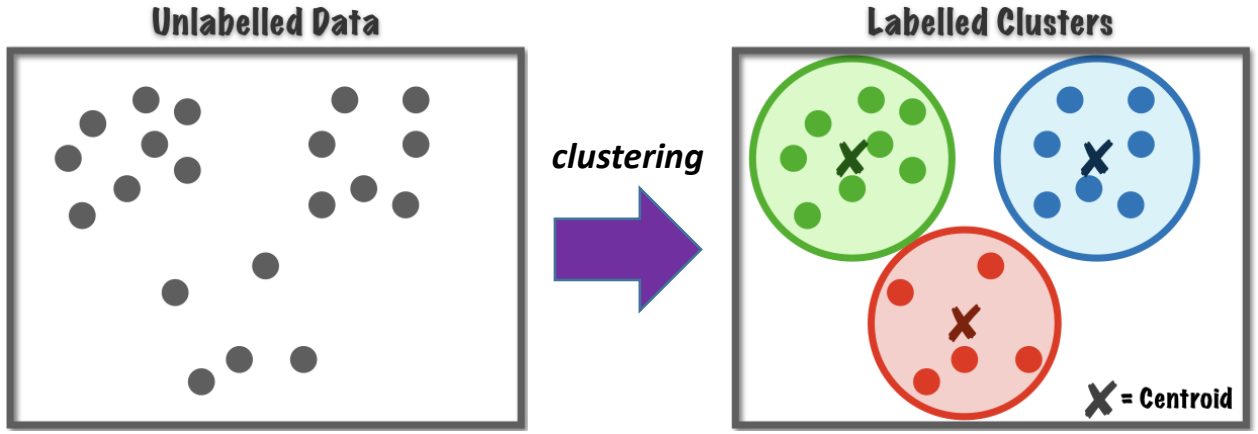
Motivação



- O objetivo do *clustering* é descobrir como os **dados se organizam naturalmente**, identificando:
  - padrões escondidos;
  - grupos de dados com características semelhantes;
  - relações e semelhanças que não são evidentes inicialmente.

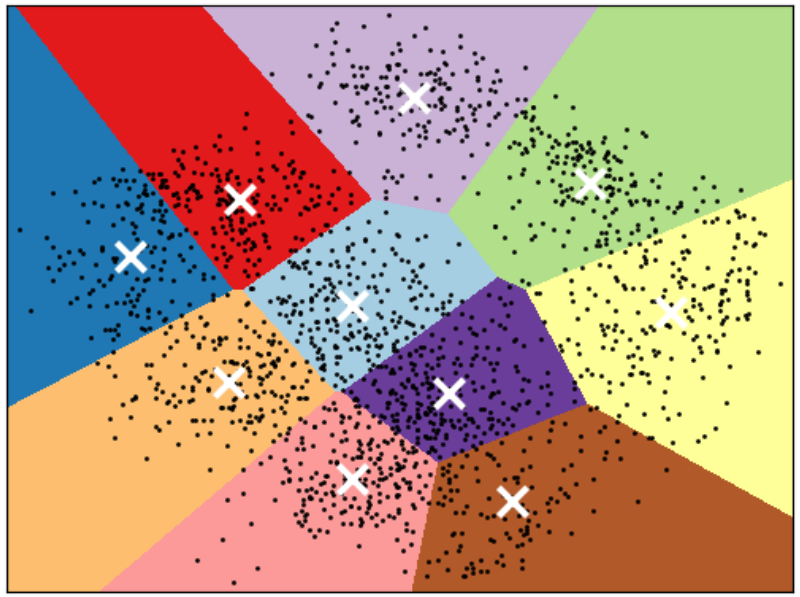
Identificação de *clusters*

- Uma das **tarefas fundamentais** do **aprendizado não-supervisionado** é a **identificação de *clusters***.
- **Entrada**: **vetores de atributo** (i.e., exemplos), **mas sem rótulos**.
- **Saída**: **representação dos *clusters* encontrados** e a **indicação de quais exemplos pertencem a cada *cluster***.
  - *k-Means*: a saída são os **centroides** e o **índice do *cluster*** de cada exemplo.



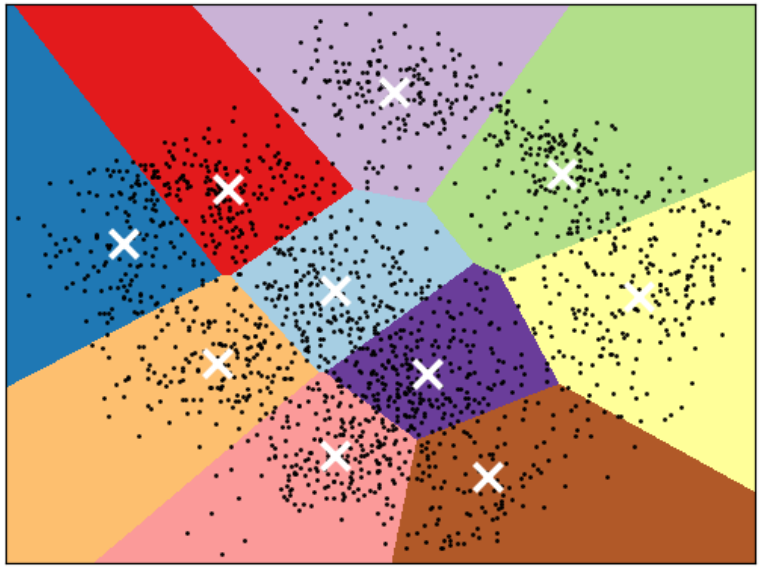
OBS.: Em 2D ou 3D dá para agrupar os dados visualmente. Porém, com 4 ou mais dimensões isso não é mais possível, então dependemos de algoritmos de *clustering* para agrupar os dados.

Aplicações: compressão e redução de dimensionalidade



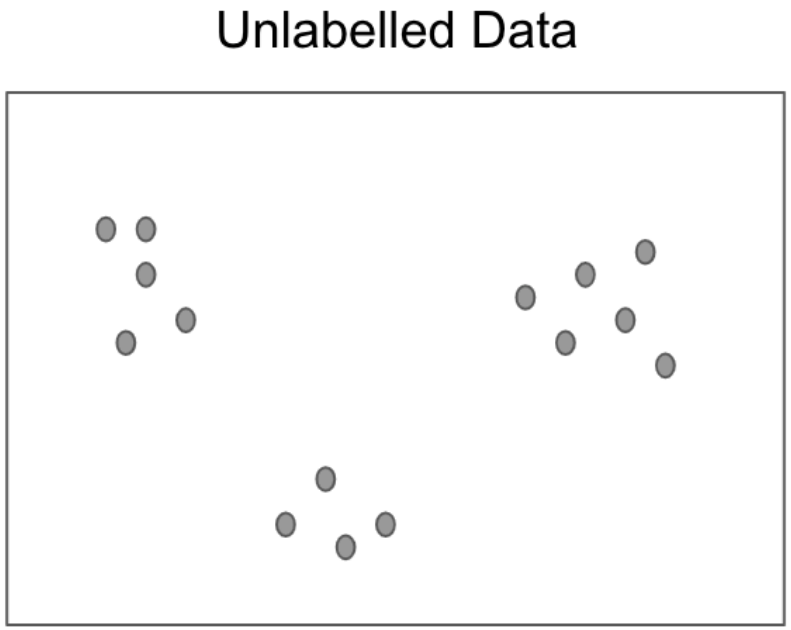
- **Compressão**: em vez de guardar todos os exemplos de um conjunto muito grande, **guardamos as representações dos *clusters*** e **a indicação do *cluster* correspondente a cada exemplo**.
  - Isso é uma compressão **com perdas**.
  - Exemplo: reduzir o número de cores de uma imagem.
- **Nova representação:** um vetor de atributos muito longo (alta dimensão) pode ser **substituído por um vetor de distâncias a k centroides**, que tem apenas k componentes.

Aplicações: anomalias e pré-processamento



- **Detecção de padrões raros (*outliers*)**
  - a área do *cluster* define o comportamento “normal”
  - tudo que estiver fora é considerado suspeito
- **Pré-processamento de dados para outros modelos**
  - criar novos atributos
    - e.g., o índice do *cluster* e as distâncias até os *clusters*
  - inicializar modelos
    - Vários algoritmos (e.g., GMM) precisam de centros iniciais.
  - identificar subgrupos para modelos especializados
    - separar os dados em grupos com características semelhantes;
    - treinar um modelo específico para cada grupo

Como representar os *clusters*?

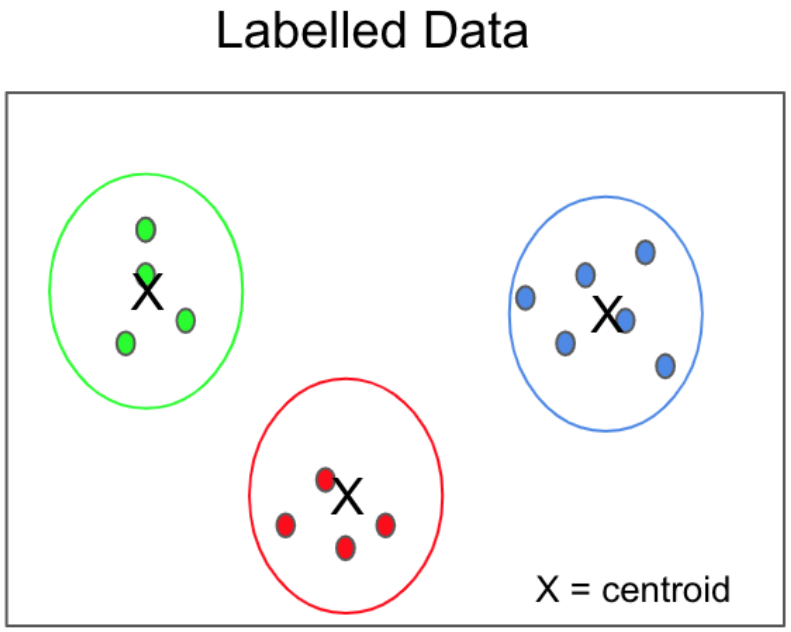


- Para identificar *clusters*, temos que primeiro **decidir como eles serão representados**.
- Existem algumas opções como a **densidade**, **distribuição**, **hierarquia**, etc.
- Porém, a abordagem mais simples e conhecida usa os **centroides** (i.e., centros) dos *clusters*.
- O algoritmo ***k-Means*** usa esta abordagem.
- Vamos ver então suas características e como funciona.

Centroide

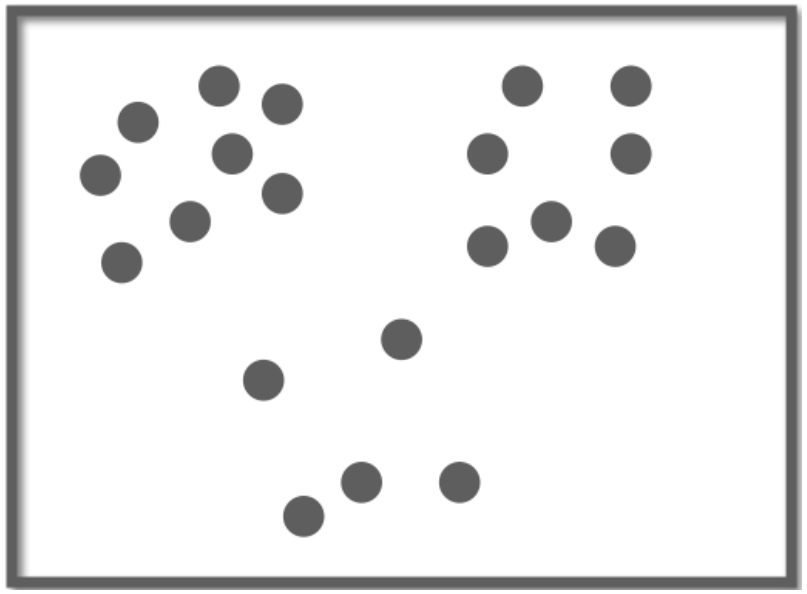
- **Centroide** é o **ponto central de um *cluster***.
  - É a **"média" das amostras que pertencem a ele**.
- Usos dos **centroides**
  - inicializar GMMs, redes neurais Bayesianas ou de funções de base radial (RBF),
  - estimar valores de atributos ausentes (imputação).
  - comprimir dados guardando só o índice do centroide mais próximo.
  - *feature engineering:* o índice do centroide e a distância até os centroides viram novos atributos.
  - detectar anomalias: distância grande até o centroide indica *outlier* ou evento raro.
  - rotular exemplos sem classes em problemas semi-supervisionados: agrupar exemplos sem classes, verificar as classes de exemplos mais representativos de cada *cluster* e rotular o restante dos exemplos com essas classes.

Como representar os *clusters*?



- Vamos supor que os seguintes **vetores de atributos bidimensionais** pertencem ao mesmo *cluster*: (2, 5), (1, 4), (3, 6).
- Assim, o **centroide** deste *cluster* é representado pela média dos vetores: (2, 5).
  - A média do primeiro atributo é $\frac{2+1+3}{3} = 2$.
  - A média do segundo atributo é $\frac{5+4+6}{3} = 5$.
- **OBS.:** O k-*Means* **exige atributos numéricos**, porque precisa calcular médias e distâncias.
  - Para atributos categóricos existem o k-*modes* e k-*prototypes*.

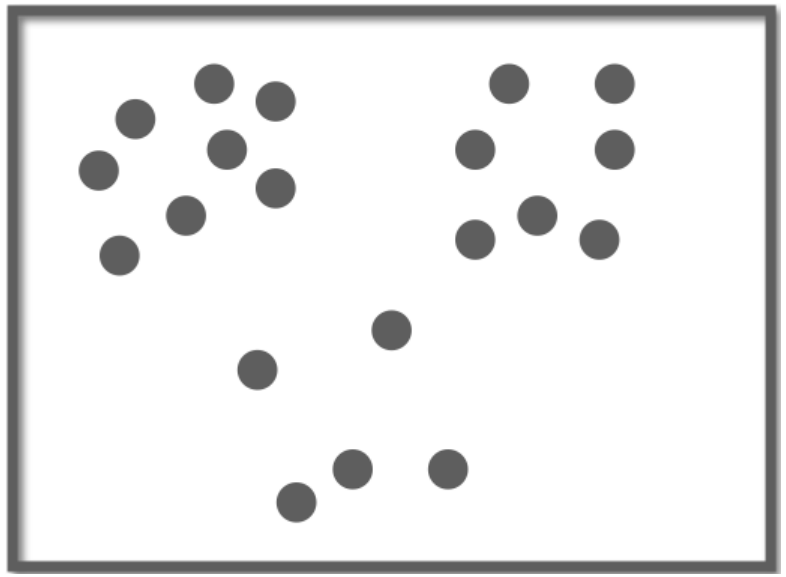
Quantos devem ser os *clusters*?



- *Clusters* **não se sobrepõem**.
- **Cada amostra pertence a um e apenas um *cluster***.
- **Dentro de um *cluster***, as **amostras ficam próximas entre si** e **distantes das amostras de outros *clusters***.
- Aí surge uma dúvida:

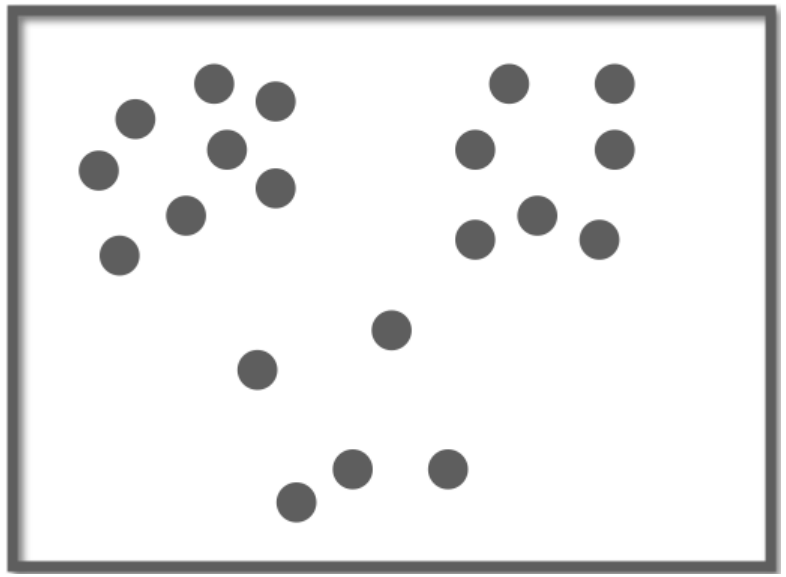
  Quantos *clusters* um conjunto de dados contém?

Quantos devem ser os *clusters*?



- Na figura, podemos identificar visualmente três ou quatro *clusters*, mas outras respostas são possíveis:
  - em um extremo, todo o conjunto de dados pode ser interpretado como um grande *cluster*;
  - no outro extremo, cada exemplo pode ser visto como representando seu próprio *cluster* de um único exemplo.

Quantos devem ser os *clusters*?



- **Implementações** práticas geralmente evitam esse problema **pedindo** ao **usuário que forneça o número de *clusters*, k**, que é um **hiperparâmetro**.
- Existem diversos métodos para **otimizar esse hiperparâmetro**, mas, dentre eles, o mais usado é o **método do cotovelo**.
  - Outros: coeficiente de silhueta e *gap statistic.*
- Veremos como ele funciona em breve.
- Porém, antes, precisamos entender como os *clusters* são criados.

Medindo distâncias

- O algoritmo precisa de uma **métrica para medir a distância entre uma amostra e o centroide** de um *cluster*.
- A métrica usada com o k-*Means* é a **distância euclidiana:**

$$d(\boldsymbol{x}, \boldsymbol{y}) = \sqrt{\sum_{i=1}^{K} (x_i - y_i)^2},$$

onde K é o número de atributos e x_i e y_i são os i-ésimos elementos dos vetores.

- **OBS.:** K define o número de atributos e k define o número de *clusters.*

A qual *cluster* um exemplo pertence?

- Vamos supor que existam k *clusters* cujos centroides são denotados pelos vetores, $\boldsymbol{\mu}_i, \forall i \in \{1, \ldots, k\}$.
- Um exemplo $\boldsymbol{x}$ tem uma certa distância $d(\boldsymbol{x}, \boldsymbol{\mu}_i)$ para cada centroide.
- Se $d(\boldsymbol{x}, \boldsymbol{\mu}_p)$ é a menor dessas distâncias, então, $\boldsymbol{x}$ é atribuído ao *cluster* correspondente ao centroide $\boldsymbol{\mu}_p$.
- Portanto, escolhemos a **menor distância** para **definir a qual *cluster* um exemplo pertence**.
- Agora, veremos como funciona o algoritmo k-*Means*.

Pseudocódigo do algoritmo *k-Means*

Entradas: conjunto de exemplos e número de *clusters*, k.

1. **Defina** k centroides iniciais (geralmente, são exemplos escolhidos do *dataset* de forma aleatória).
2. **Repita**
   a) Calcule a distância de cada exemplo, $\boldsymbol{x}$, para cada um dos k centroides.
   b) Atribua cada exemplo, $\boldsymbol{x}$, ao *cluster* do centroide mais próximo.
   c) Calcule o novo centroide de cada *cluster**.
3. **Enquanto** as posições dos centroides continuarem mudando.

*os novos centroides são calculados como a média aritmética (centro geométrico) dos exemplos pertencentes a cada *cluster*.

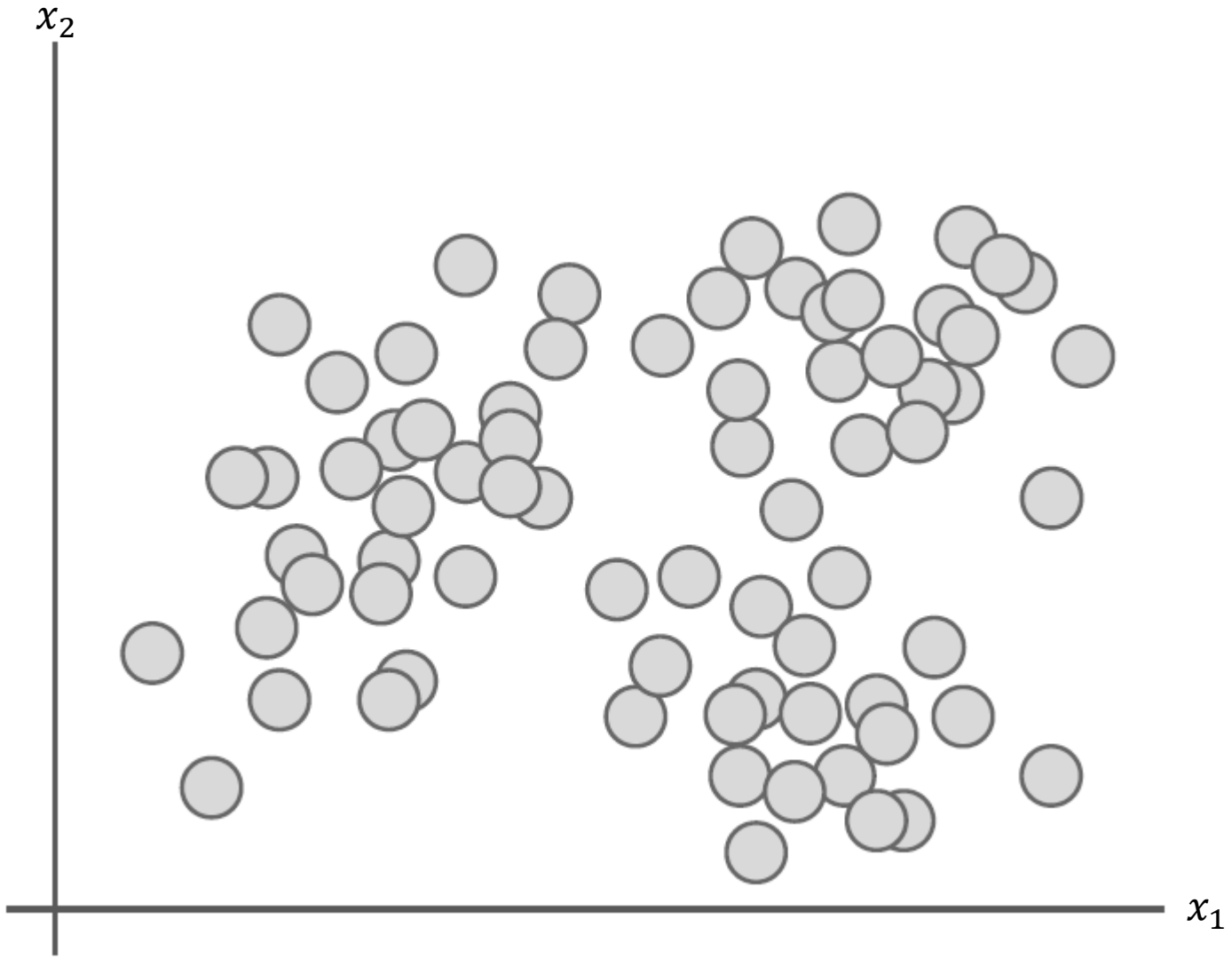
- O algoritmo **converge em um número finito de iterações** para uma situação em que **cada exemplo se encontra no *cluster* mais próximo**, de modo que, a partir deste momento, os **centroides não mudam mais**.
- Não há garantia que ele convirja para o mínimo global.

Vejamos um exemplo

Funcionamento do *k-Means*

Quais são os *clusters* para os dados ao lado?

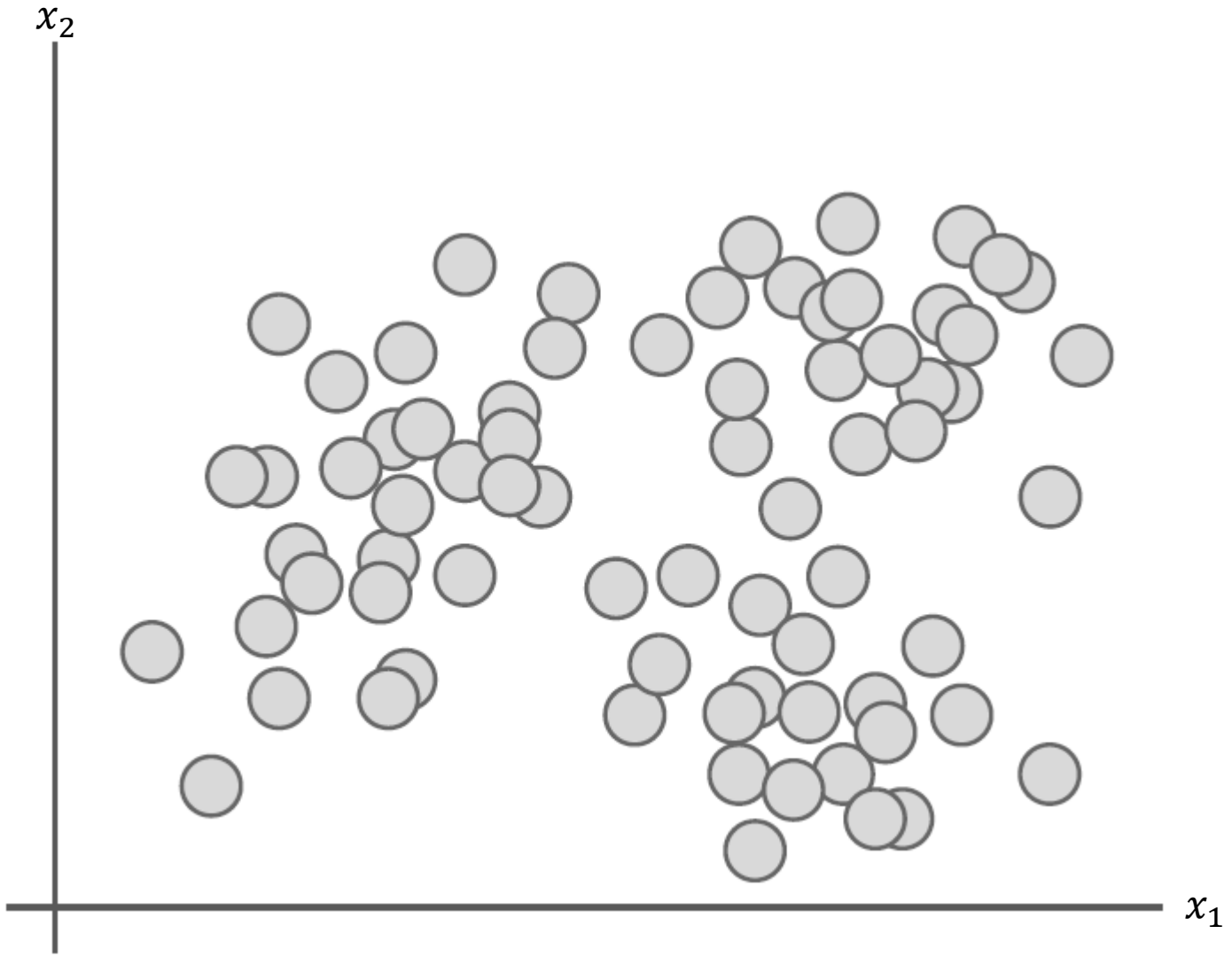
Visualmente, **parece haver três grupos**, mas o algoritmo ainda não sabe disso.



Funcionamento do *k-Means*

1. Definimos o número de *clusters*, k = 3

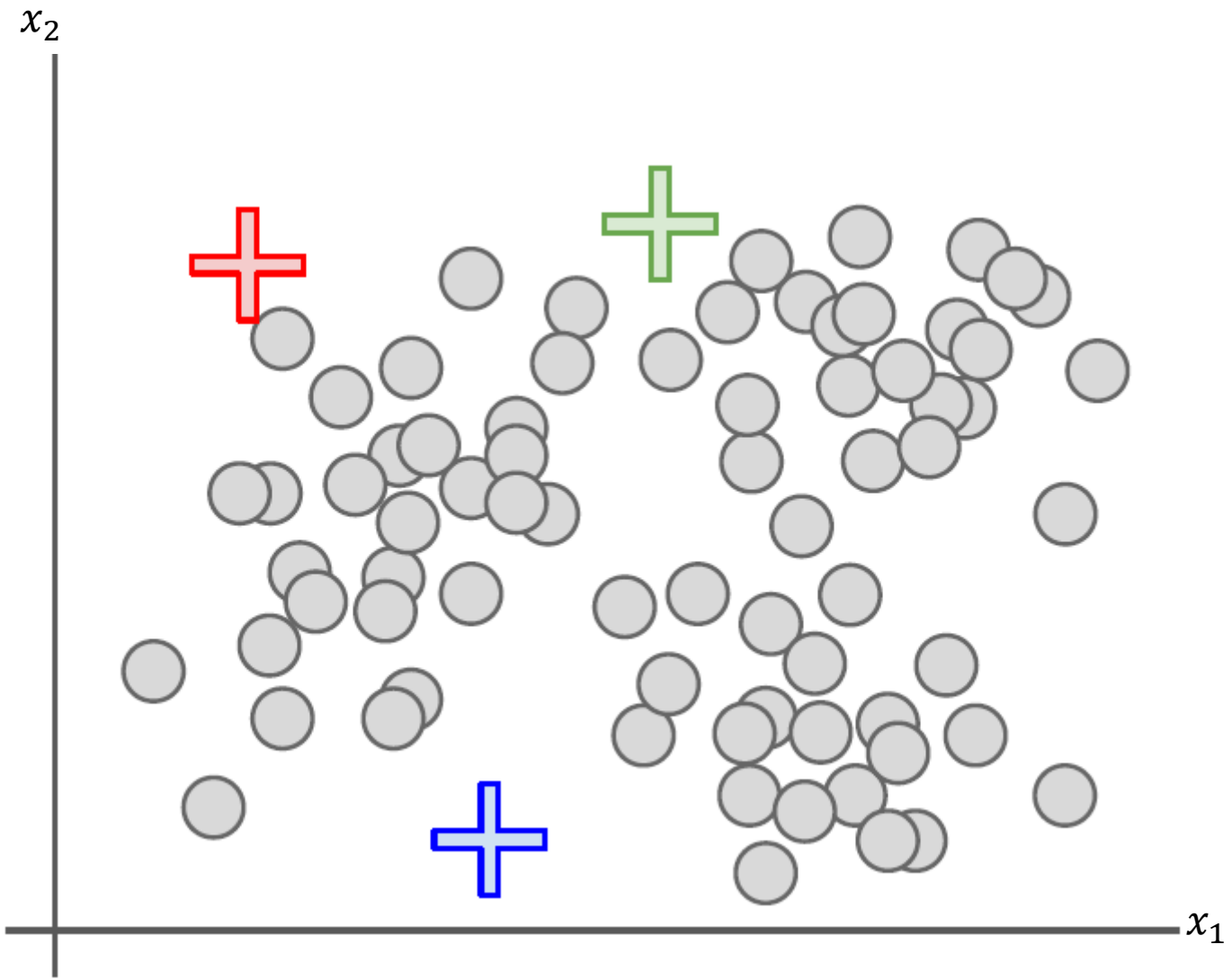
- k = 3 pode não ser o valor ideal.
- Para encontrar o valor ideal, pode-se usar o método do cotovelo.



Funcionamento do *k-Means*

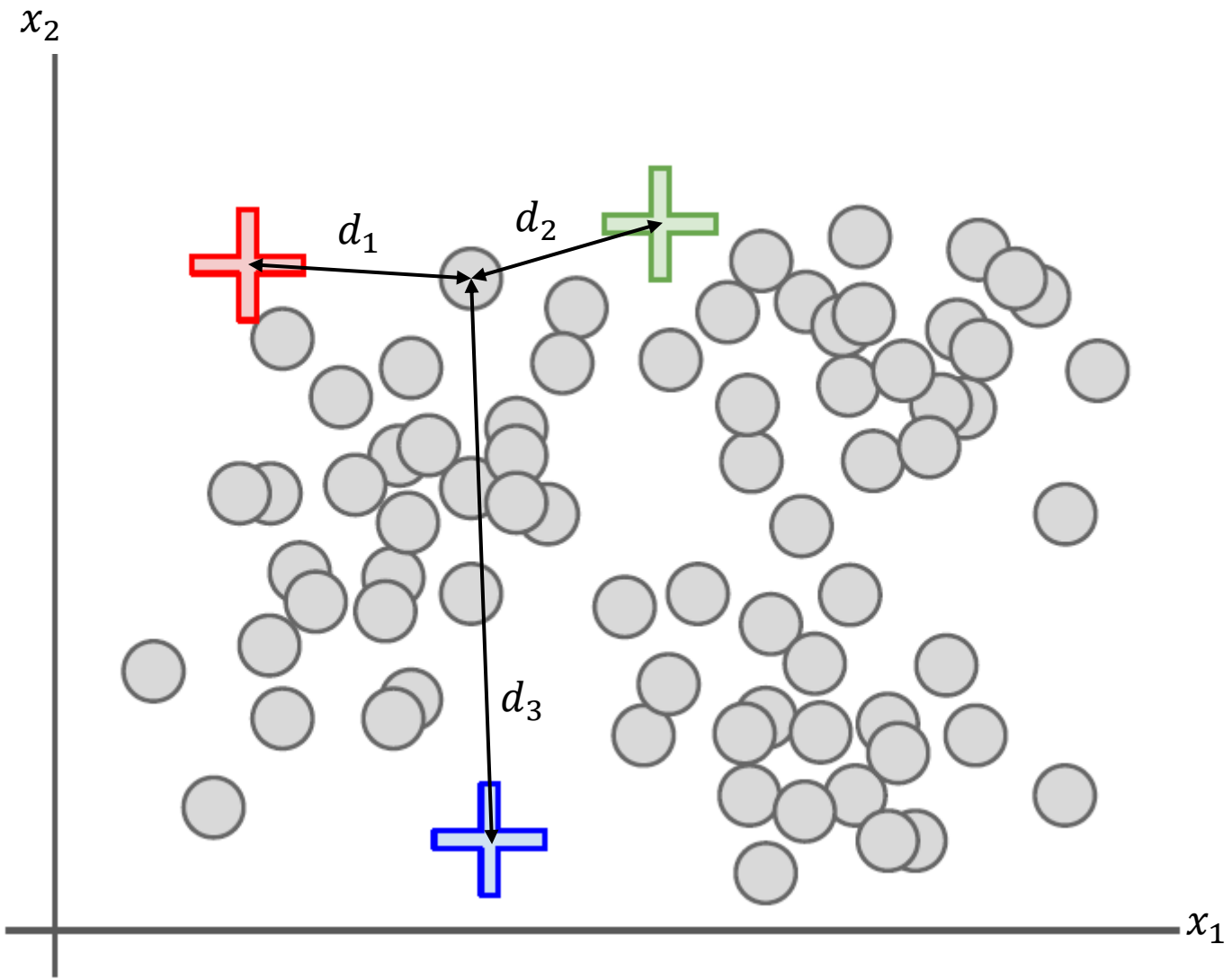
2. Inicializa-se **aleatoriamente** os k = 3 centroides (cruzes coloridas) de cada *cluster.*

- No caso desse exemplo, os centroides são vetores bidimensionais.



Funcionamento do *k-Means*

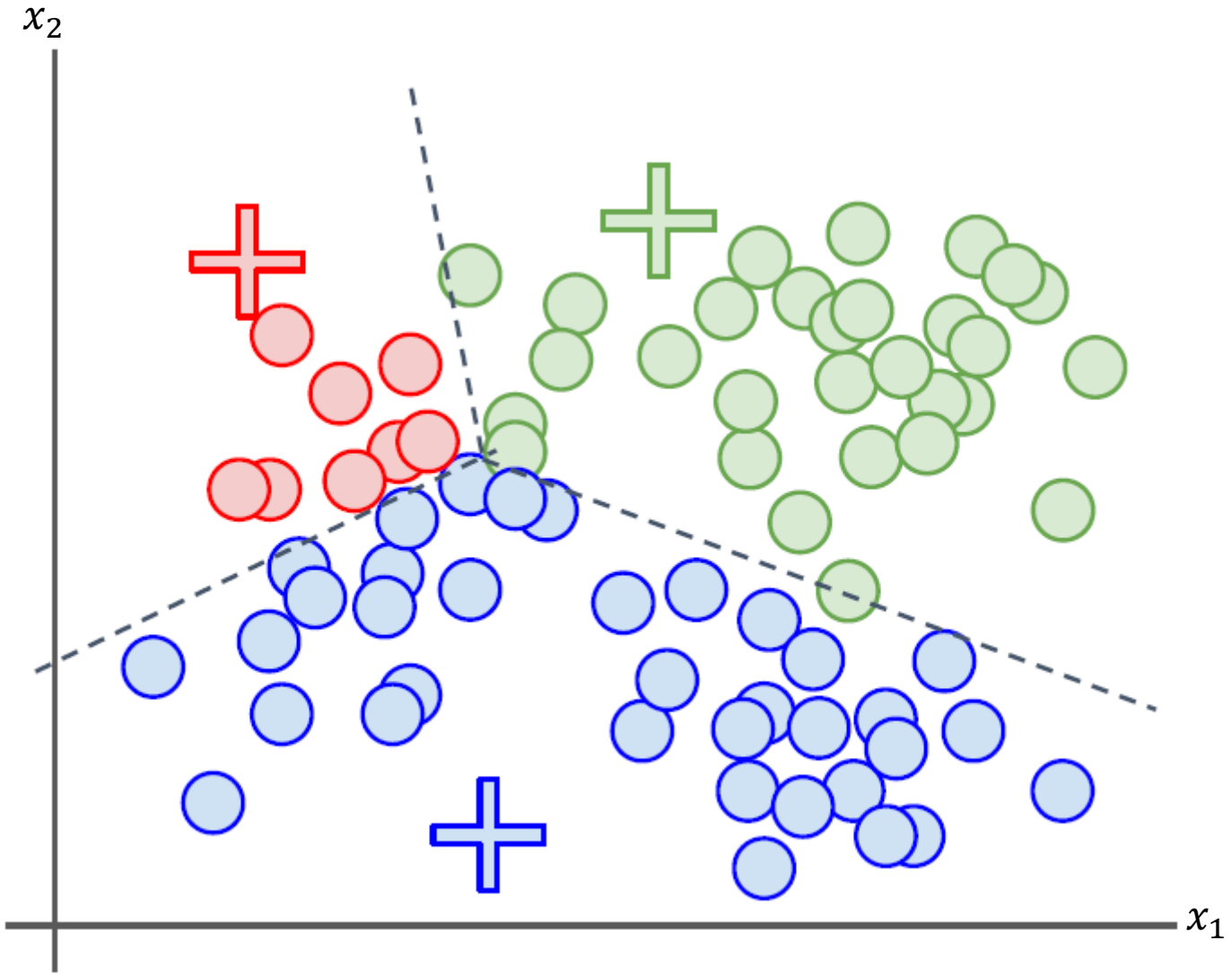
3. Calcula-se a **distância** de cada exemplo (círculos cinzas) para cada um dos 3 centroides.



Funcionamento do *k-Means*

4. Atribui-se cada exemplo ao centroide mais próximo com base na distância euclidiana.

Cada centroide define uma região.

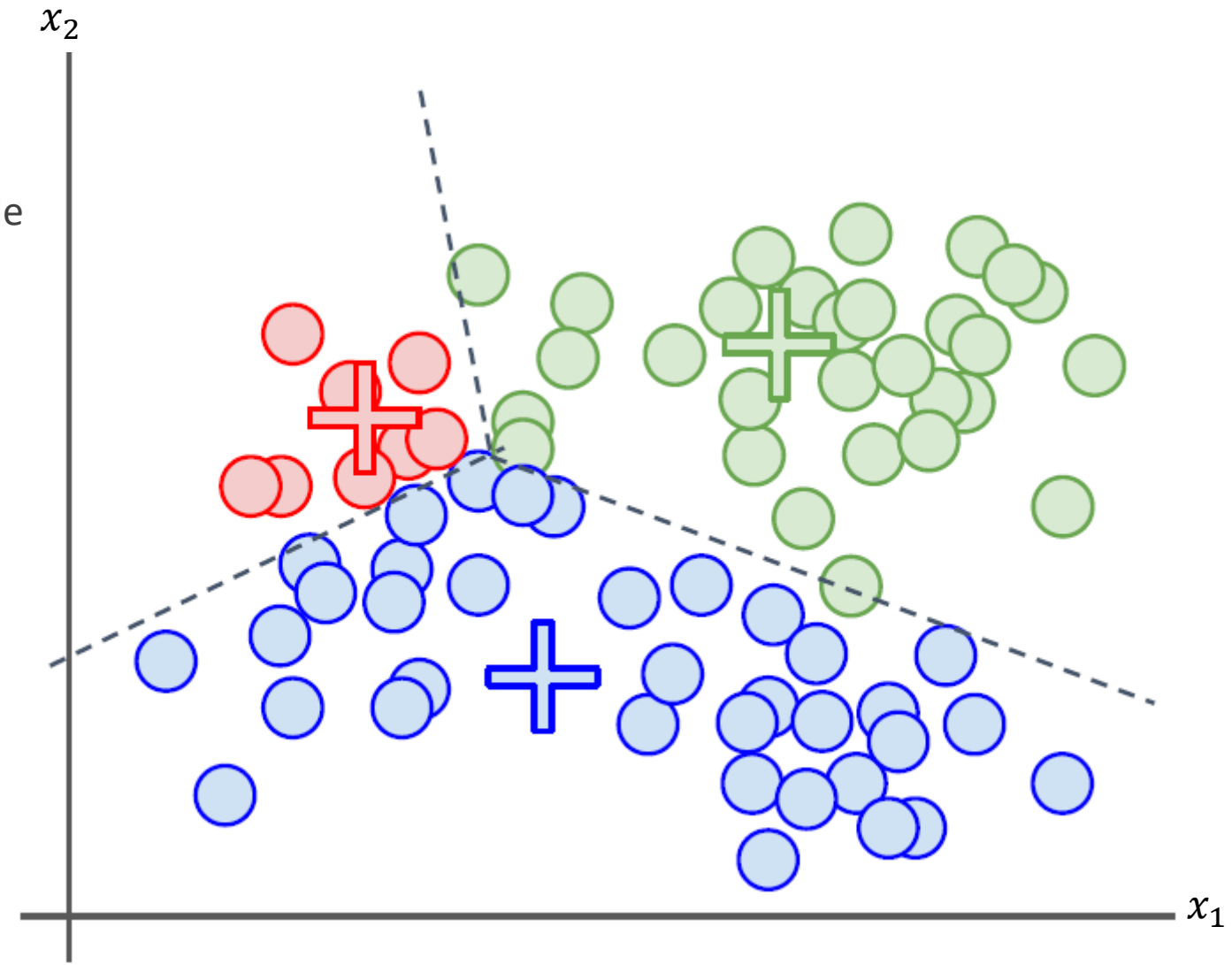


Funcionamento do *k-Means*

5. Calcula-se o novo centroide de cada *cluster* usando-se a média dos exemplos que pertencem a ele.

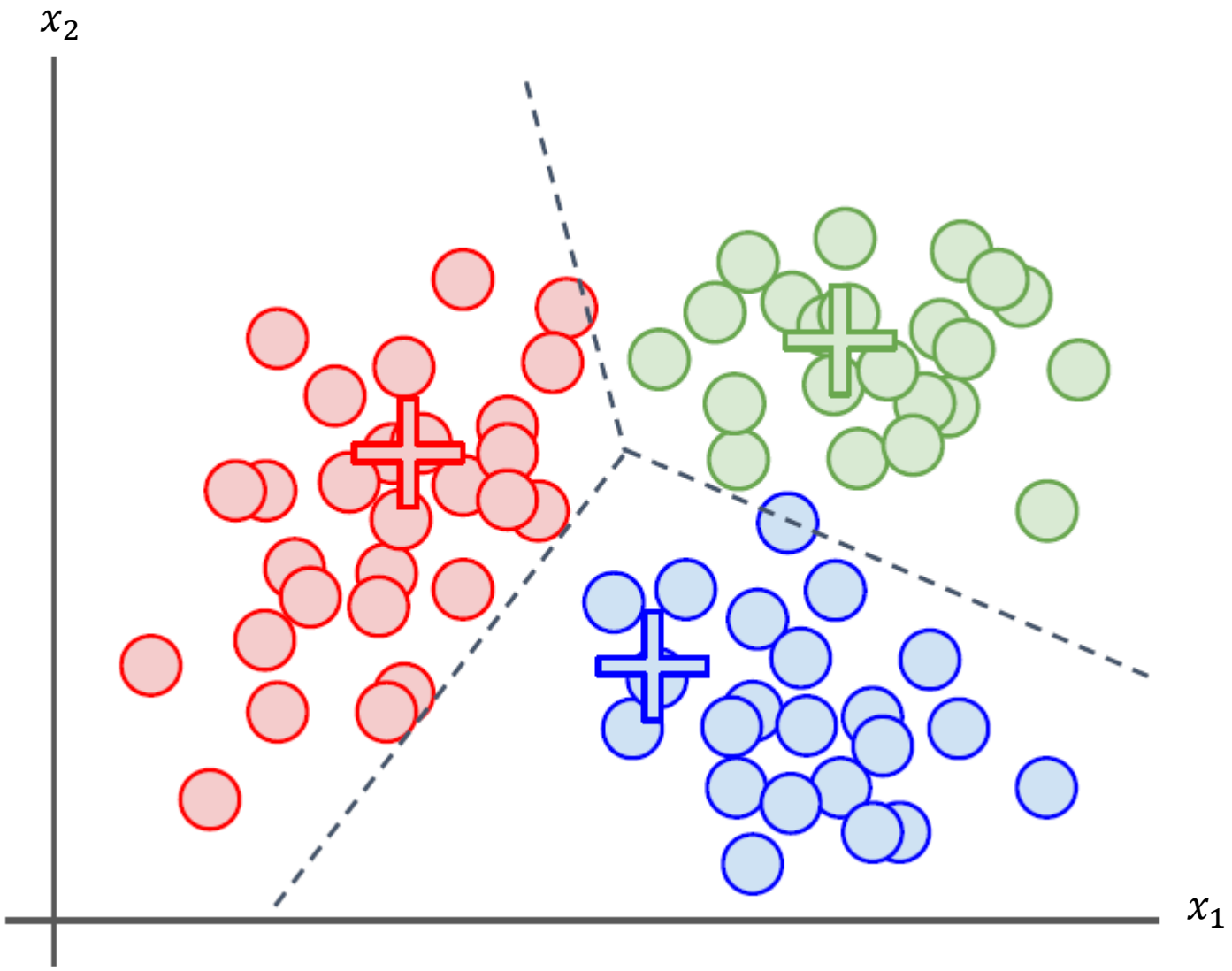
Os centroides são os **centros geométricos** de cada *cluster*.

OBS.: Reparem que as cruzes se deslocaram para o “meio” dos pontos de cada cor.



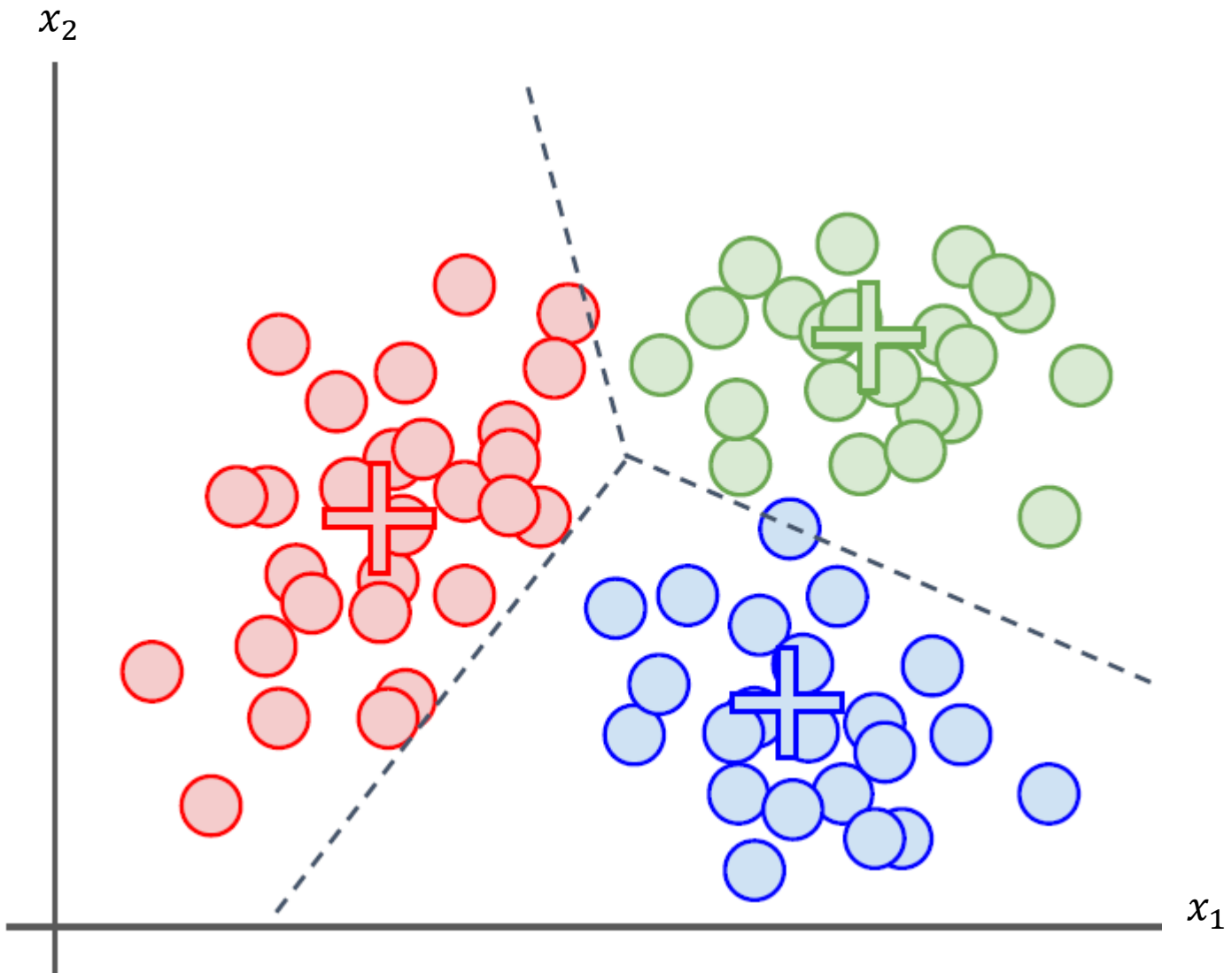
Funcionamento do *k-Means*

6. Repete-se os passos 3, 4 e 5.



Funcionamento do *k-Means*

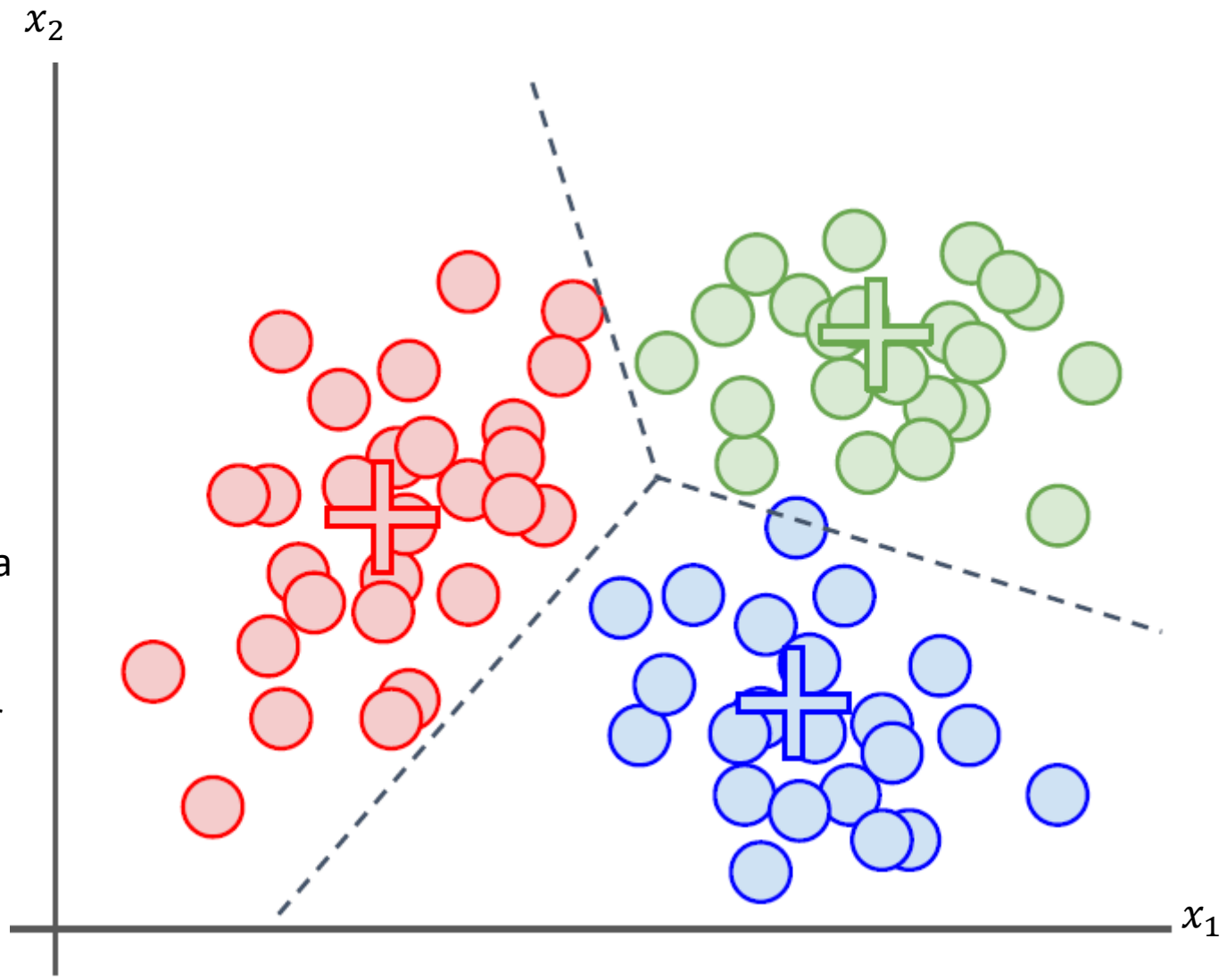
7. Repete-se os passos 3, 4 e 5.



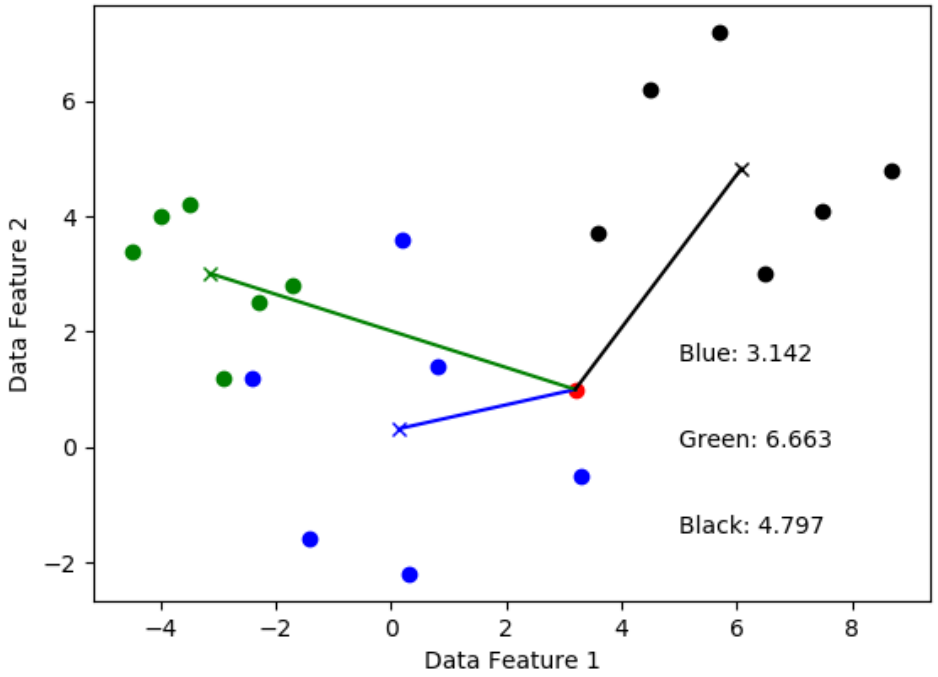
Funcionamento do *k-Means*

Repete-se os passos 3 à 5 até que um dos casos abaixo ocorra:

1. Os centroides não mudam mais (ou mudem muito pouco, abaixo de uma tolerância),
2. A soma das **distâncias entre os exemplos e seus centroides** seja minimizada (e permaneça praticamente constante),
   - Distancia *intra-cluster*
3. O número máximo de iterações seja atingido.

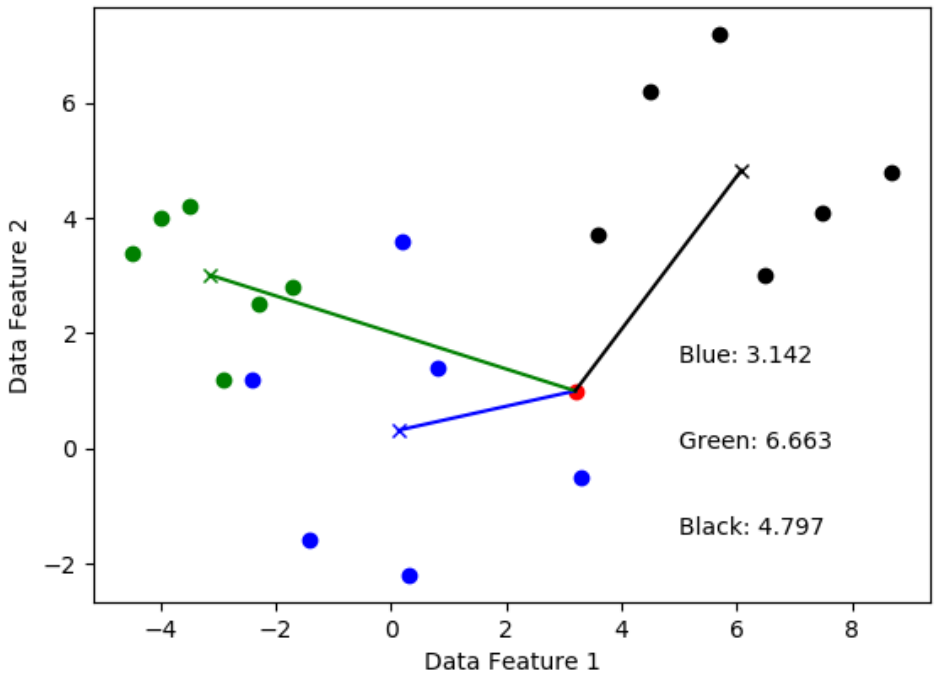


Como inicializar os centroides?



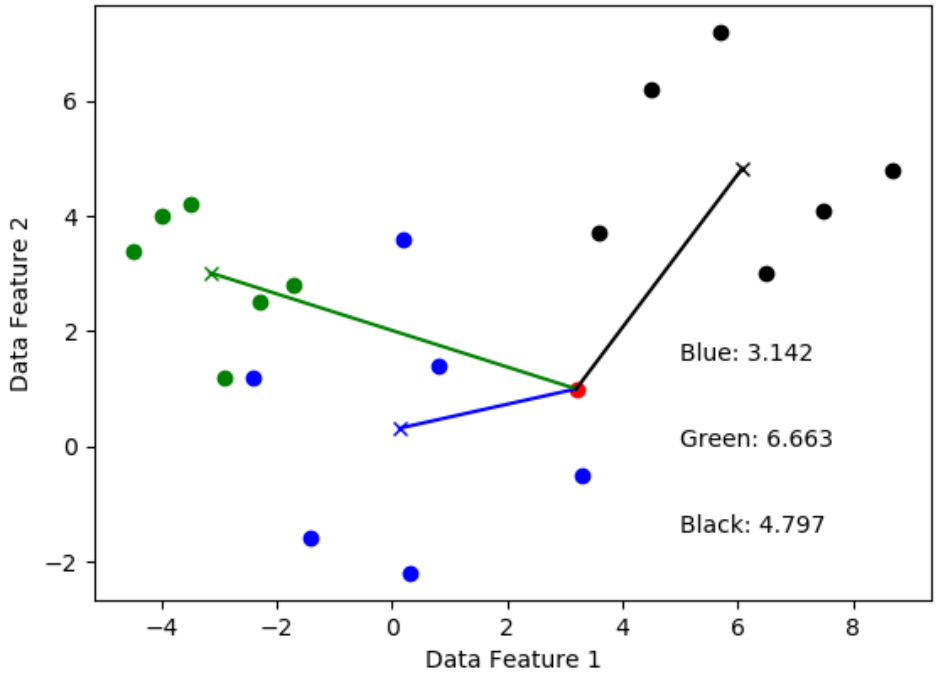
- Além da **inicialização com valores aleatórios**, pode-se **escolher aleatoriamente k exemplos de treinamento** como os centroides iniciais.
- Porém, existem alguns problemas.
- Se os centroides iniciais mudam:
  - os *clusters* iniciais podem mudar completamente
  - fazendo com que o algoritmo convirja para ***clusters* diferentes a cada execução**.
- Ou seja, **cada inicialização aleatória** pode resultar em **centroides finais diferentes**.
  - O desempenho do modelo pode variar com a inicialização.

Como inicializar os centroides?



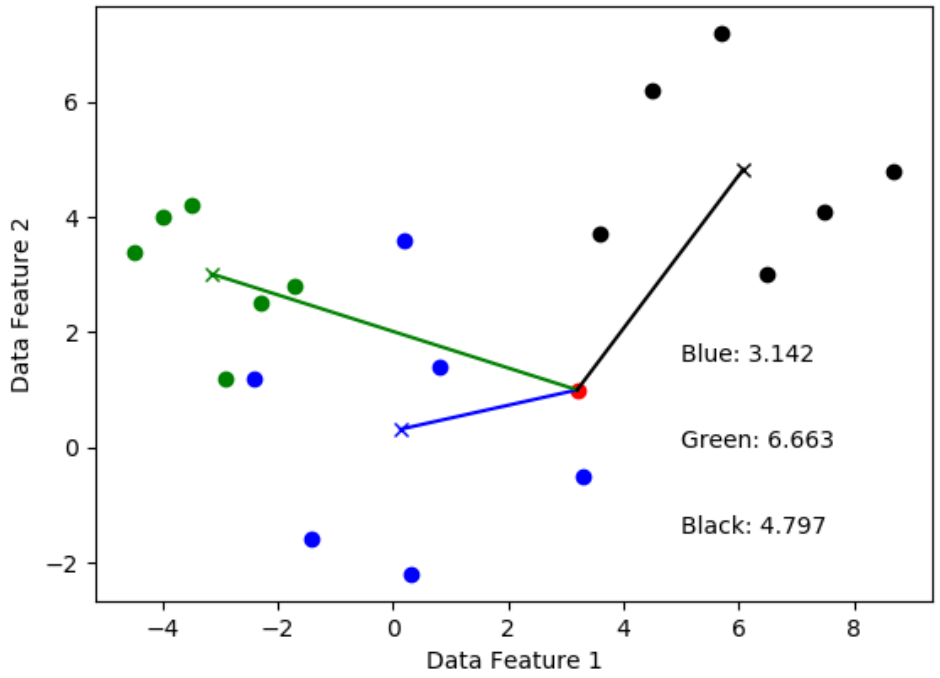
- O número de **transferências** de exemplos **entre *clusters* depende dos centroides iniciais**.
- Se os **centroides iniciais** forem **muito próximos entre si**, podem haver **muitas transferências**, elevando o tempo de treinamento.
- Por outro lado, se os **centroides iniciais** já forem **perfeitos**, **nenhum exemplo é transferido para outro *cluster*** e o algoritmo é encerado.

Como inicializar os centroides?



- Portanto, a **inicialização é muito importante**, pois **um bom ponto de partida acelera a convergência**.
- Ou seja, o **tempo de convergência e o resultado final** dependem da inicialização.
- **Conclusão**: O *k-Means* é **altamente sensível à escolha inicial dos centroides.**

Como inicializar os centroides?

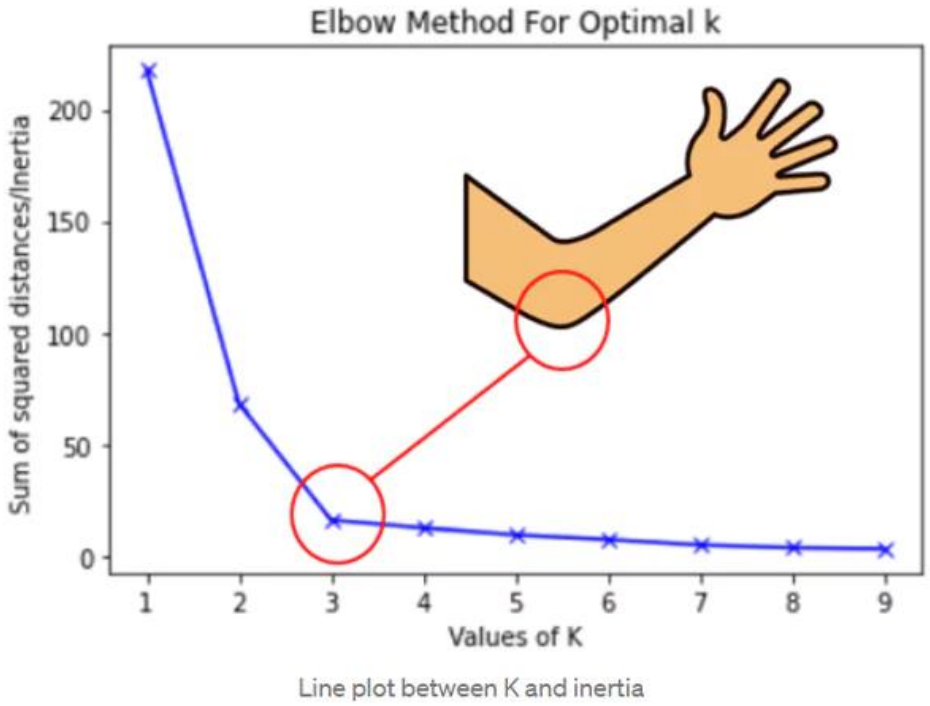


- Em resumo, todo o processo **depende do ponto de partida**.
- Portanto, como encontramos os melhores *clusters*?

Rodamos o algoritmo várias vezes, com **inicializações aleatórias diferentes**, e ficamos com o **melhor resultado**, ou seja, o que **minimiza a distância intra-*cluster*** (também chamada de **inércia**).

Como definimos o número de *clusters*?

Método do cotovelo



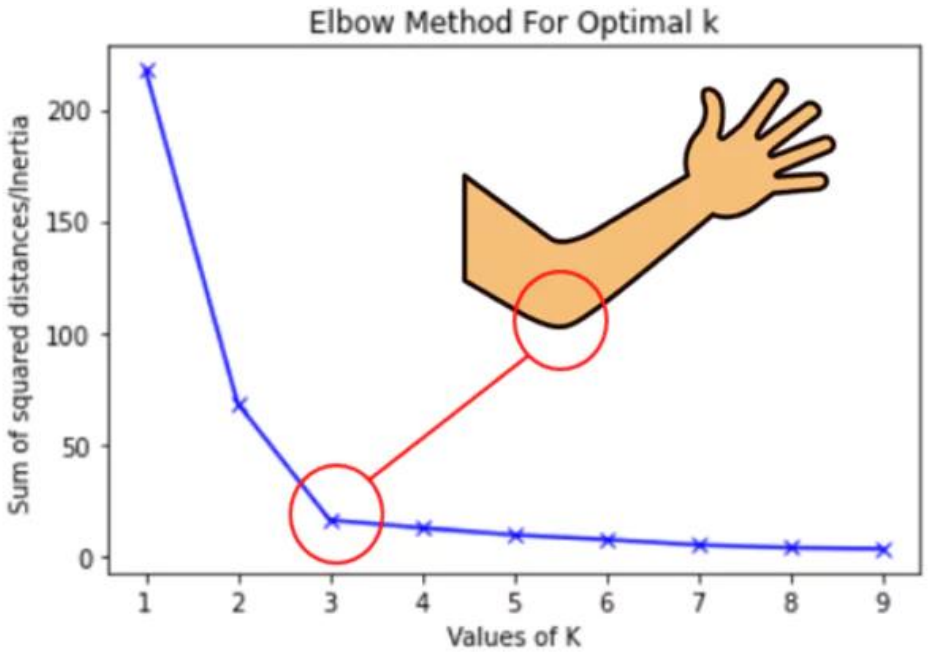
Line plot between K and inertia

- O método calcula a **soma dos quadrados das distâncias intra-*clusters*** (**inércia**) para vários valores de k

$$I(k) = \sum_{i=1}^{k} \sum_{\boldsymbol{x} \in C_i} \|\boldsymbol{x} - \boldsymbol{\mu}_i\|^2 ,$$

onde $\boldsymbol{\mu}_i$ é o centroide do i-ésimo *cluster,* C_i é conjunto de exemplos pertencentes ao i-ésimo *cluster* e $\|\boldsymbol{x} - \boldsymbol{\mu}_i\|^2$ é a distância euclidiana ao quadrado entre $\boldsymbol{x}$ e $\boldsymbol{\mu}_i$.

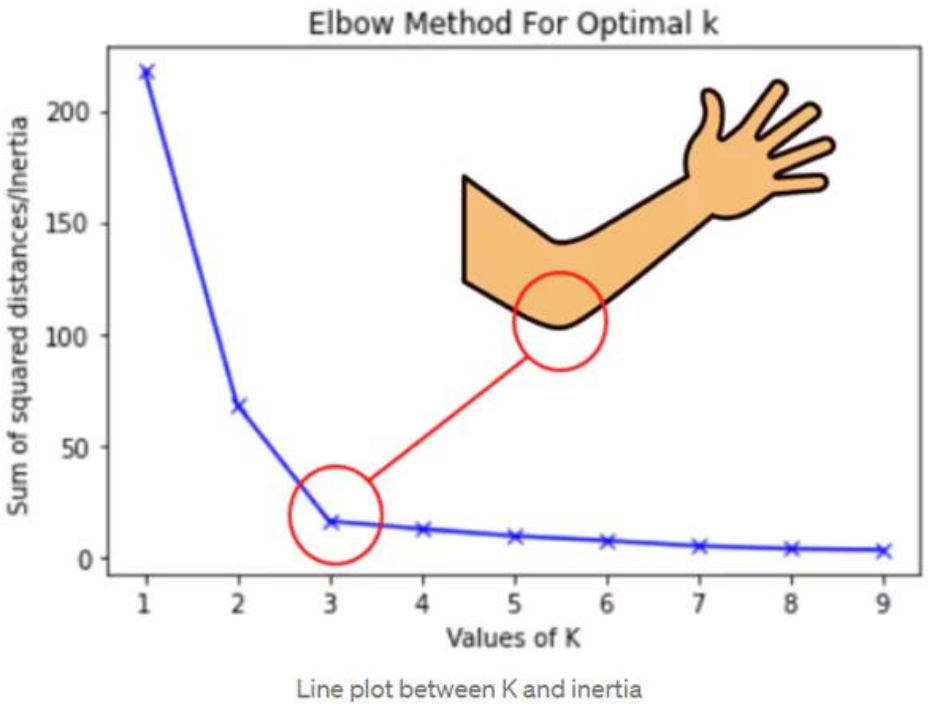
Soma dos quadrados intra-*clusters* (inércia)



Line plot between K and inertia

- Ela mede o quão "**compactos**" são os *clusters*.
  - quanto menor → mais próximo os exemplos estão entre si e de seus centroides.
- No limite, quando $k = N$ (número de amostras), cada exemplo é seu próprio *cluster* e a inércia é 0 (mínima possível).
- Porém, quanto maior k, maior a chance do **modelo se tornar extremamente complexo** e se **sobreajustar** aos dados de treinamento.

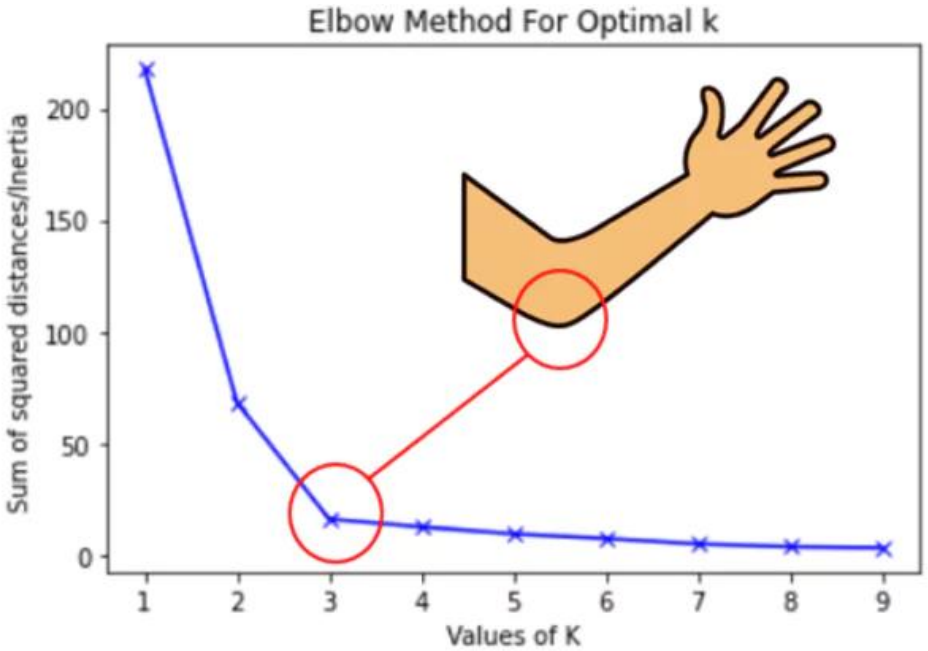
Método do cotovelo



Line plot between K and inertia

- Assim, aumentar k **sempre reduz (ou mantém) a inércia**, mas isso não significa que o agrupamento fique mais útil.
- O objetivo é encontrar o valor de k onde **a melhoria trazida pelo aumento de *clusters* passa a ser pequena**.
- Esse ponto é chamado de "**cotovelo**"**.**
- Por isso **não escolhemos o k de menor inércia**, e sim o do cotovelo.
  - Então como encontramos o cotovelo?

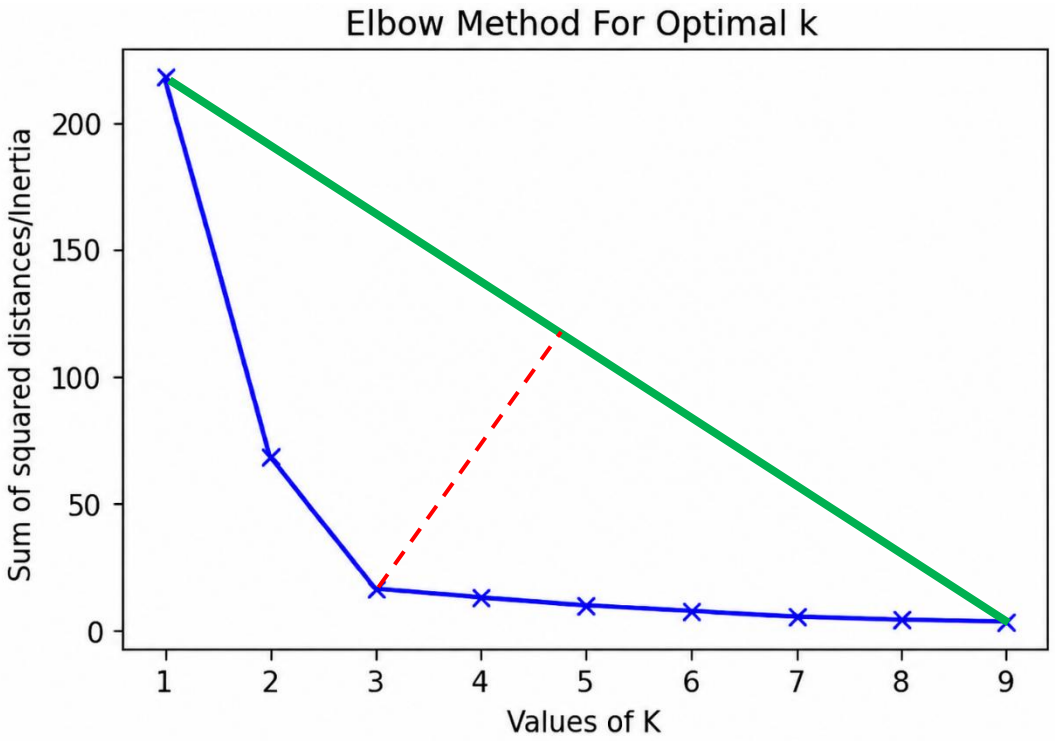
Soma dos quadrados intra-*clusters* (inércia)



Line plot between K and inertia

- Calcula-se e plota-se o **valor da inércia para um intervalo** de valores de k.
- A curva resultante quantifica **o quão próximos os exemplos de um *cluster* estão do seu centroide**.
- Assim, o valor ideal de k é encontrando localizando-se o **cotovelo na curva**.

Soma dos quadrados intra-*clusters* (inércia)

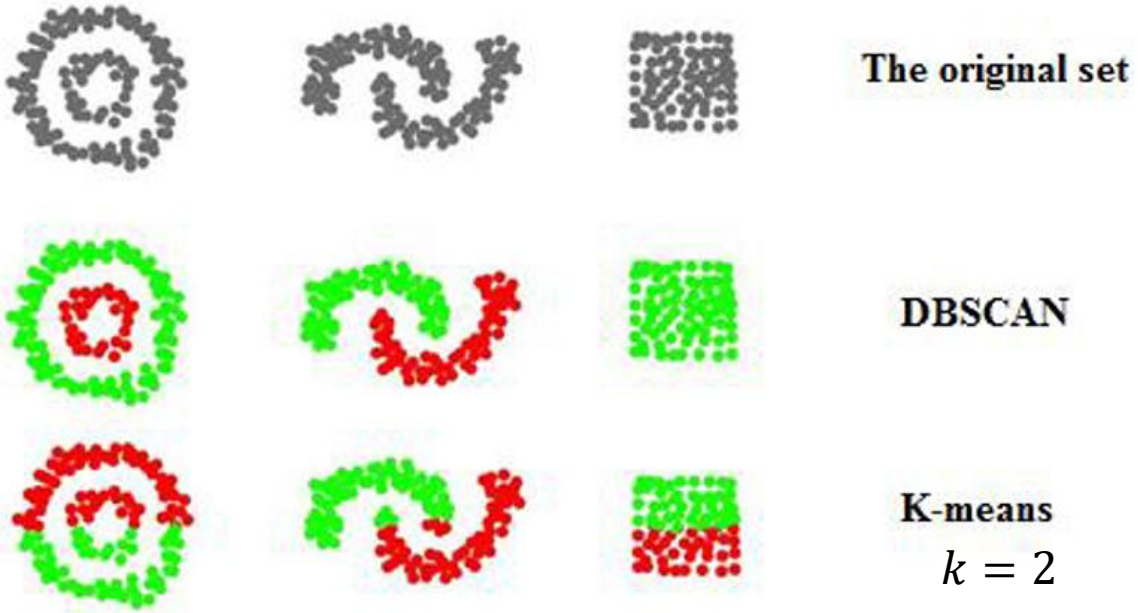


- Se o cotovelo **não for nítido** (curva suave), uma técnica comum é:
  - traçar uma **reta ligando o primeiro e o último ponto** da curva;
  - escolher o ponto da curva **mais distante dessa reta** como cotovelo.
- É uma **heurística**: ajuda a decidir, mas não é uma prova de que esse k dá o número ótimo de *clusters*.

Outros algoritmos de *clustering*

- *Density-based spatial clustering of applications with noise* (DBSCAN): agrupa por **densidade**, enquanto o k-*Means* usa **centróides.**
- **Não** exige definir o número de *clusters* previamente.
- Ele busca **regiões de alta densidade** para criar os *clusters*.
- Possui dois parâmetros:
  - ε (epsilon): raio da vizinhança, o qual define o que é “perto”;
  - MinPts: número mínimo de exemplos dentro desse raio para a região ser considerada densa.
- Exemplos que não pertencem a nenhuma região densa são rotulados como **ruído** (*outlier*).
- **Limitação**: um único ε raramente funciona bem quando os ***clusters* têm densidades diferentes**.

Outros algoritmos de *clustering*



- Enquanto o k-*Means* responde a pergunta

  Onde k centroides devem ficar para **minimizar a distância intra-*cluster***?

- O DBSCAN responde a pergunta:

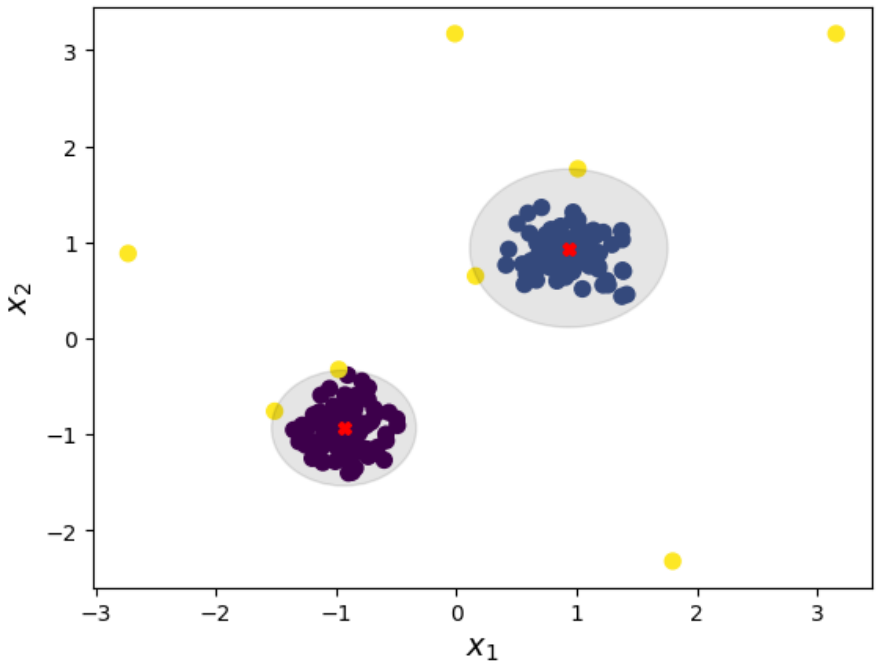
  Quais regiões do espaço têm **densidade suficiente** para serem **consideradas *clusters***?

- ***k-Means*** funciona bem com ***clusters* aproximadamente esféricos** (compactos e de tamanhos parecidos), já o **DBSCAN detecta formas arbitrárias**.

Outros algoritmos de *clustering*

- HDBSCAN (*Hierarchical* DBSCAN): É uma extensão do DBSCAN que constrói uma **hierarquia de densidades.**
  - Reduz o problema do ε único do DBSCAN usando uma faixa de valores de densidade.
  - Procura regiões densas em múltiplos níveis de densidade e seleciona os agrupamentos mais estáveis.
- Misturas de Gaussianas (GMMs): Supõem que os dados foram gerados por uma mistura de **distribuições gaussianas**, cada uma **representando um *cluster***.
  - Método probabilístico que dá a **probabilidades de pertencimento dos exemplos a cada *cluster***.
- Documentação da biblioteca *scikit-learn* sobre clusterização:
  - https://scikit-learn.org/stable/modules/clustering.html

Aplicação: detecção de anomalias com k-*Means*



- Nessa aplicação, encontramos os **centroides e a área dos *clusters***.
- A **área** de cada *cluster* é definida por um **raio.**
- O raio é escolhido de modo que uma porcentagem dos exemplos (em geral $\geq$ **95%**) fique a uma distância **menor ou igual a ele** do centroide mais próximo.
- Calcula-se um raio por *cluster*, como na figura.
- Uma amostra é considerada uma **anomalia** se a **distância entre ela e o centroide** mais próximo **for maior do que o raio**.
  - Ou seja, se ela estiver fora da área do *cluster*.

Exemplo

- Exemplo: kmeans.ipynb



Lista de exercícios

Lista #13 – kmeans

Perguntas?

Obrigado!





k-means be like:



K-MEANS CLUSTER



